

App-Telemetrie konfigurieren

Wie Traces, Logs und Metriken einer App strukturiert und dauerhaft in Unity Catalog gespeichert werden können.

App-Telemetrie ist eine Funktion (aktuell in "Public Preview", also einer öffentlichen Testphase), mit der Traces (Ablaufverfolgungen einzelner Anfragen), Logs und Metriken einer App dauerhaft in Unity-Catalog-Tabellen gespeichert werden können. Grundlage dafür ist das offene Protokoll **OpenTelemetry (OTel)**, ein Industriestandard zur Erfassung solcher Beobachtungsdaten. Das System erfasst automatisch System-Logs und Nutzungsereignisse (z. B. Nutzer-Login, direkte API-Anfragen); zusätzlich lässt sich eigene Instrumentierung (Code zur gezielten Datenerfassung) einbauen.

Voraussetzungen

Regionale Verfügbarkeit

Der Workspace muss sich in einer der unterstützten Regionen befinden: `ap-northeast-1`, `ap-northeast-2`, `ap-south-1`, `ap-southeast-1`, `ap-southeast-2`, `ca-central-1`, `eu-central-1`, `eu-west-1`, `eu-west-2`, `sa-east-1`, `us-east-1`, `us-east-2` oder `us-west-2`.

Berechtigungen

Um neue Ziel-Tabellen für die Telemetrie zu erstellen, sind `CAN MANAGE`-Berechtigungen auf Katalog und Schema sowie `CREATE TABLE` auf das Schema nötig. Um in bereits existierende Tabellen zu schreiben, benötigt man entweder `CAN MANAGE`-Berechtigungen, oder alle Account-Nutzer müssen `USE CATALOG`, `USE SCHEMA`, `SELECT` und `MODIFY` auf die Ziel-Tabellen besitzen.

Anforderungen an die Tabellen

Ziel-Tabellen müssen verwaltete ("managed") Delta-Tabellen sein und sich in derselben Region wie der Workspace befinden. Databricks empfiehlt, "Predictive Optimization" (automatische Leistungsoptimierung) auf den Telemetrie-Zieltabellen zu aktivieren, um die Abfrageleistung zu verbessern.

Telemetrie aktivieren

Über die Benutzeroberfläche

1. Zur App-Detailseite navigieren
2. Im Reiter "Settings" den Bereich "App telemetry" öffnen
3. Auf "Add" klicken und Katalog sowie Schema angeben
4. Es werden automatisch drei Tabellen erstellt: `otel_metrics`, `otel_spans` und `otel_logs`
5. Optional lässt sich ein Tabellenpräfix angeben, sodass die Tabellen als `<prefix>_otel_metrics`, `<prefix>_otel_spans` und `<prefix>_otel_logs` benannt werden
6. Auf "Save" klicken und die App neu bereitstellen, damit die Telemetriedaten zu fließen beginnen

Über die CLI

Alternativ lässt sich der Befehl `databricks apps update` mit einer JSON-Konfiguration nutzen, die die vollständigen Katalog-, Schema- und Tabellennamen für `logs_table`, `metrics_table` und `traces_table` angibt. Danach erfolgt das erneute Deployment mit `databricks apps deploy`.

Telemetriedaten überprüfen

Nach dem erneuten Deployment füllt sich die Tabelle `otel_logs` automatisch. Die Tabellen `otel_spans` und `otel_metrics` füllen sich hingegen erst, nachdem eigene Instrumentierung hinzugefügt wurde.

Vorgehen zur Überprüfung:

1. Die App-URL aufrufen, um Aktivität zu erzeugen
2. Einige Sekunden warten, bis der erste Datenstapel (Batch) ankommt
3. Eine SQL-Abfrage ausführen: `SELECT * FROM <catalog>.<schema>.otel_logs LIMIT 10;`

Telemetriedaten abfragen

Wichtige Spalten zum Filtern und Verknüpfen von Daten sind `time`, `service_name`, `trace_id`, `span_id` und `attributes`. Die Spalte `attributes` enthält ereignisspezifische

Metadaten wie `event.name` .

Das vollständige Tabellenschema lässt sich mit folgendem Befehl anzeigen:

```
DESCRIBE TABLE <catalog>.<schema>.otel_logs;
```

Beispiel: Fehler-Logs abfragen

Die Dokumentation liefert eine Beispielabfrage für Fehler-Logs der letzten Stunde – nützlich zur Fehlersuche. Die Abfrage filtert nach `service_name` , `severity_text = "ERROR"` und dem Zeitraum, sortiert absteigend nach Zeit.

System-Ereignisse abfragen

Databricks erfasst automatisch System-Ereignisse in `otel_logs` , z. B. Nutzungsereignisse. Diese lassen sich über das Attribut `event.name` filtern. Ein Beispiel ruft die 100 neuesten Nutzungsereignisse einer App ab, indem nach `event.name = 'app.auth'` gefiltert wird.

Eigene Instrumentierung (Custom Instrumentation)

Für verschiedene Frameworks gibt es konkrete Anleitungen zur Einbindung von OpenTelemetry:

Framework	Vorgehen
Streamlit	Startbefehl in <code>app.yaml</code> auf <code>['opentelemetry-instrument', 'streamlit', 'run', 'app.py']</code> setzen; Umgebungsvariable <code>OTEL_TRACES_SAMPLER</code> auf <code>always_on</code> setzen; Pakete wie <code>opentelemetry-distro</code> , <code>opentelemetry-exporter-otlp-proto-grpc</code> , <code>opentelemetry-instrumentation-tornado</code> und <code>opentelemetry-instrumentation-system-metrics</code> hinzufügen
Dash	Startbefehl auf <code>opentelemetry-instrument python app.py</code> setzen; Pakete wie <code>dash</code> , <code>dash-bootstrap-components</code> , <code>pandas</code> , <code>plotly</code> , <code>databricks-sql-connector</code> , <code>databricks-sdk</code> , <code>python-dotenv</code> , <code>dash-ag-grid</code> sowie OpenTelemetry-Pakete hinzufügen
Flask	Startbefehl auf <code>['opentelemetry-instrument', 'flask', '--app', 'app.py', 'run', '--no-reload']</code> setzen; Pakete <code>opentelemetry-distro</code> , <code>opentelemetry-exporter-otlp-proto-grpc</code> und <code>opentelemetry-instrumentation-flask</code> hinzufügen

FastAPI	Startbefehl auf <code>['opentelemetry-instrument', 'uvicorn', 'app:app', '--host', '0.0.0.0', '--port', '8000']</code> setzen; <code>fastapi</code> , <code>uvicorn</code> sowie passende OpenTelemetry-Pakete hinzufügen
Node.js	Eine Datei <code>otel.js</code> anlegen, die einen NodeSDK mit <code>OTLPTraceExporter</code> , <code>OTLPMetricExporter</code> (Export-Intervall 10 Sekunden) und <code>OTLPLogExporter</code> konfiguriert; <code>package.json</code> um Start-Skript und Abhängigkeiten für <code>express</code> , <code>morgan</code> sowie die entsprechenden OpenTelemetry-SDK-Pakete ergänzen

Umgebungsvariablen

Databricks konfiguriert in der App-Laufzeitumgebung automatisch Umgebungsvariablen für den OTLP-Collector-Endpunkt, das Exportprotokoll, Ressourcen-Attribute und die Batch-Verarbeitung. Eine vollständige Übersicht findet sich in der Dokumentation zu den Umgebungsvariablen der App-Telemetrie (siehe auch Seite "Apps-Umgebung").

Kosten

Die Telemetrie-Erfassung wird als Lakeflow-Connect-Nutzung über den "Zerobus Ingest"-Connector abgerechnet – getrennt von den Compute-Kosten der App. Die Abrechnung erfolgt über die SKU "Jobs Serverless". Kosten lassen sich über die Systemtabelle zur abrechenbaren Nutzung mit Filter auf "Zerobus Ingest" nachverfolgen. Die reinen App-Compute-Kosten werden separat über das App-Kosten-Monitoring erfasst (siehe Seite "Apps überwachen").

Limits und Einschränkungen

App-Telemetrie nutzt den Zerobus-Ingest-Connector; alle dessen Einschränkungen gelten entsprechend, etwa bezüglich Datensatzgröße, Durchsatz und Zustellgarantien. Zusätzlich gilt eine **maximale Log-Zeilengröße von 1 MB pro Log-Zeile**.

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/observability>
Stand: 2026-08-06